

PROYECTO 1: Emulador de CPU Segmentada (Pipeline de 5 Etapas)

Contexto Arquitectónico

Una CPU segmentada (*pipelined*) ejecuta instrucciones solapadas en 5 etapas:

1. **IF (Fetch)**: trae la instrucción de la memoria de programa.
2. **ID (Decode)**: decodifica el opcode y lee los registros origen.
3. **EX (Execute)**: ejecuta la operación en la ALU.
4. **MEM (Memory)**: lee o escribe en RAM.
5. **WB (Write Back)**: escribe el resultado en el banco de registros.

Riesgos de Datos (Data Hazards)

Cuando una instrucción en ID necesita un dato que una instrucción anterior todavía no escribió, el hardware resuelve el conflicto de dos formas:

- **Forwarding (Anticipación)**: desvía el dato directamente desde el registro de segmentación EX/MEM o MEM/WB hacia la entrada de la ALU, sin esperar a que pase por WB.
- **Stall (Burbuja)**: si la instrucción productora es un LOAD, el dato no está disponible hasta que sale de MEM. En ese caso **no existe otra opción** que congelar el pipeline un ciclo e insertar un NOP (burbuja) en EX. Esto se llama **Load-Use Hazard**.

Especificación de Datos

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Registro {
    R0, R1, R2, R3,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Instruccion {
    NOP,
```

```

    ADD { dest: Registro, src1: Registro, src2: Registro },
    SUB { dest: Registro, src1: Registro, src2: Registro },
    LOAD { dest: Registro, direccion_ram: u8 },
    STORE { src: Registro, direccion_ram: u8 },
}

/// Estado físico de un registro de segmentación entre dos etapas del pipeline.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct RegistroSegmentacion {
    pub instruccion: Instruccion,
    pub activa: bool,

    /// Resultado calculado por la ALU (o el dato leído de RAM), disponible
    /// recién después de que la instrucción pasa por su etapa correspondiente.
    /// `None` mientras el resultado todavía no fue calculado.
    pub resultado: Option<u16>,
}

pub struct CpuSegmentada {
    pub if_id: RegistroSegmentacion,
    pub id_ex: RegistroSegmentacion,
    pub ex_mem: RegistroSegmentacion,
    pub mem_wb: RegistroSegmentacion,
    pub registros: [u16; 4],
    pub program_counter: usize,
    pub contador_ciclos: u64,
}

```

Nota de diseño respecto a un skeleton previo de este mismo enunciado: si ya intentaste una versión anterior de este ejercicio, revisá con cuidado qué le pasa a la instrucción LOAD en el ciclo exacto donde se detecta un stall. Es un punto donde es fácil, sin querer, perder una instrucción del pipeline por completo. Antes de seguir, escribí (sección de Tests, más abajo) el test `test_stall_no_pierde_la_instruccion_load` y hacé que pase — es requisito obligatorio, no opcional.

Tareas a Implementar

Tarea 1.1 — Unidad de Detección de Riesgos

```
impl CpuSegmentada {  
    /// Determina si la instrucción actualmente en ID depende de un LOAD  
    /// que está en este momento en la etapa EX (registro id_ex).  
    /// Retorna `true` si hay un Load-Use Hazard y se requiere un stall.  
    pub fn detectar_load_use_hazard(&self, instruccion_en_id: &Instruccion) -> bool  
    {  
        todo!()  
    }  
}
```

Requisitos:

- Debe cubrir los 4 tipos de instrucción que pueden estar en id_ex y en instruccion_en_id, usando match exhaustivo (no if let suelto — practicá la exhaustividad del Módulo 3).
- Un LOAD en id_ex genera hazard con un ADD/SUB en ID si su dest coincide con src1 o src2.
- Un LOAD en id_ex genera hazard con un STORE en ID si su dest coincide con el src del STORE.

Tarea 1.2 — Unidad de Anticipación (Forwarding)

```
impl CpuSegmentada {  
    /// Busca si el valor de `src` puede anticiparse desde una etapa posterior  
    /// del pipeline, en vez de leerse del banco de registros.  
    /// Prioridad: EX/MEM tiene prioridad sobre MEM/WB (el dato más reciente gana).  
    pub fn calcular_forwarding(&self, src: Registro) -> Option<u16> {  
        todo!()  
    }  
}
```

Requisitos:

- Revisa primero `self.ex_mem`; si esa instrucción escribe en `src` y su resultado ya es `Some`, devuélvelo.
- Si no, revisa `self.mem_wb` bajo el mismo criterio.
- Si ninguna aplica, devuélve `None` (el llamador deberá leer `self.registros[...]` directamente).
- Solo `ADD`, `SUB` y `LOAD` son instrucciones que "escriben" un registro destino — `STORE` y `NOP` nunca son fuente de forwarding.

Tarea 1.3 — Resolución de Operandos (lectura con forwarding aplicado)

```
impl CpuSegmentada {  
    /// Devuelve el valor a usar para `src`, aplicando forwarding si corresponde,  
    /// o leyendo el banco de registros si no hay nada que anticipar.  
    pub fn resolver_operando(&self, src: Registro) -> u16 {  
        todo!()  
    }  
}
```

Tarea 1.4 — Etapa EX real (ALU)

```
impl CpuSegmentada {  
    /// Ejecuta la operación de la ALU para la instrucción que está en id_ex,  
    /// y devuelve el `RegistroSegmentacion` resultante (con `resultado` ya  
    /// calculado)  
    /// listo para avanzar a ex_mem.  
    pub fn ejecutar_alu(&self, instruccion: RegistroSegmentacion) ->  
    RegistroSegmentacion {  
        todo!()  
    }  
}
```

Requisitos:

- `ADD/SUB`: usá `resolver_operando` para ambos operandos, calculá, y guardá el resultado en el campo `resultado`.
- `LOAD`: en esta etapa **no** hay resultado todavía (se calcula en `MEM`) — dejá `resultado`:

None y simplemente propagá la instrucción.

- STORE/NOP: propagá sin cálculo.

Tarea 1.5 — Etapa MEM real (integración con el Proyecto 2)

```
impl CpuSegmentada {  
    /// Ejecuta la etapa MEM: si la instrucción es un LOAD, lee de la memoria  
    /// (recibida como parámetro) y completa el campo `resultado`. Si es un STORE,  
    /// escribe en la memoria el valor a guardar (resuelto con resolver_operando  
    /// ANTES de que la instrucción abandone id_ex – pensá bien en qué momento  
    /// del pipeline tenés ese dato disponible).  
    pub fn ejecutar_mem(  
        &self,  
        instruccion: RegistroSegmentacion,  
        memoria: &mut crate::cache::ControladorMemoria, // ver Proyecto 2  
    ) -> RegistroSegmentacion {  
        todo!()  
    }  
}
```

Esta función es la que conecta ambos proyectos del TP. Vas a completarla recién después de tener el Proyecto 2 funcionando — dejala con todo!() hasta entonces, o hacé una versión provisoria que simule la memoria con un array [u8; 256] simple, y migrala después.

Tarea 1.6 — Etapa WB real

```
impl CpuSegmentada {  
    /// Si la instrucción en mem_wb tiene un registro destino y un resultado  
    /// calculado, escribe ese valor en el banco de registros.  
    pub fn ejecutar_writeback(&mut self) {  
        todo!()  
    }  
}
```

Requisito: R0 es de solo lectura por convención de hardware (siempre vale 0) — cualquier intento de escribirlo debe ignorarse silenciosamente, no debe hacer panic.

Tarea 1.7 — Ciclo de reloj completo

```
impl CpuSegmentada {  
    pub fn ciclo_reloj(&mut self, programa: &[Instruccion], memoria: &mut  
        crate::cache::ControladorMemoria) {  
        todo!()  
    }  
}
```

Requisitos del orden de ejecución (respétalo estrictamente, es el orden real del hardware):

1. Ejecutar WB con el contenido actual de mem_wb (Tarea 1.6).
2. Calcular ejecutar_mem sobre el contenido actual de ex_mem (Tarea 1.5) → ese resultado es lo que va a pasar a mem_wb este ciclo.
3. Detectar hazard sobre la instrucción en if_id (la que está a punto de entrar a ID→EX).
4. **Si hay hazard:** el contenido de id_ex avanza a ex_mem (ya resuelto por ALU en un ciclo anterior — no se recalcula acá), se inserta un NOP en id_ex, y **no** se avanza if_id ni program_counter. Termina el ciclo acá.
5. **Si no hay hazard:** id_ex avanza a ex_mem pasando por ejecutar_alu (Tarea 1.4). if_id avanza a id_ex. Se hace fetch de una nueva instrucción a if_id (o NOP si no queda programa).
6. Incrementar contador_ciclos.

Tarea 1.8 — Trazabilidad visual (Display)

```
impl std::fmt::Display for CpuSegmentada {  
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {  
        todo!()  
    }  
}
```

Requisito: debe imprimir una tabla de una sola fila por ciclo, mostrando qué instrucción hay en cada etapa (IF, ID, EX, MEM, WB), similar a:

```
Ciclo 3 | IF: ADD R3,R1,R2 | ID: LOAD R2,0x10 | EX: ADD R1,R0,R0 | MEM: -- | WB: --
```

Tarea 1.9 — Riesgo de Control (Stretch Goal)

Agregá una variante JUMP { direccion_destino: usize }. Cuando se resuelve en la etapa EX, si implica un salto, las instrucciones que ya entraron incorrectamente en if_id e id_ex deben descartarse (flush), reemplazándolas por NOP, y program_counter debe redirigirse a direccion_destino.

Tests Obligatorios

Implementá los siguientes tests en #[cfg(test)] mod tests. Se describe el escenario y el resultado esperado — la implementación del test (arrange/act/assert) es tuya.

1. **test_forwarding_ex_mem_tiene_prioridad:** armá un estado donde tanto ex_mem como mem_wb escriben en el mismo registro con valores distintos. calcular_forwarding debe devolver el valor de ex_mem, no el de mem_wb.
2. **test_stall_no_pierde_la_instruccion_load:** programa de 2 instrucciones — LOAD R1, 0x00 seguido de ADD R2, R1, R1. Corré ciclos hasta que el LOAD complete su WB. Verificá que self.registros[R1] tiene el valor correcto leído de memoria (no 0, no un valor perdido).
3. **test_sin_hazard_no_genera_stall:** dos instrucciones independientes (ADD R1,R2,R3 seguido de SUB R2,R0,R0, sin dependencia entre sí). Verificá que el pipeline avanza sin detenerse (program_counter avanza cada ciclo).
4. **test_r0_es_de_solo_lectura:** intentá que una instrucción escriba en R0 en la etapa WB. Verificá que self.registros[R0] == 0 después, sin panic.
5. **test_cpi_con_un_stall:** contá cuántos ciclos totales toma completar un programa de 3 instrucciones que contiene exactamente un Load-Use Hazard, y verificá que el total de ciclos es el esperado (pipeline ideal + 1 ciclo de burbuja).

PROYECTO 2: Controlador de Caché Asociativa por Conjuntos (2 Vías, LRU, Write-Back)

Contexto Arquitectónico

Simulamos una memoria RAM de 256 bytes con una caché asociativa por conjuntos de 2 vías.

Decodificación de direcciones (8 bits)

Bloques de 4 bytes ($2^2 \rightarrow 2$ bits de offset), 4 conjuntos ($2^2 \rightarrow 2$ bits de índice):

```
+-----+-----+-----+
| Tag (4b) | Index (2b) | Offset (2b) |
+-----+-----+-----+
| Bit 7...4 | Bit 3...2 | Bit 1...0 |
+-----+-----+-----+
```

Política Write-Back

La CPU escribe en la caché (rápida), marcando la línea con `dirty_bit = true`. El dato **no** se propaga a la RAM hasta que esa línea deba ser desalojada (por la política LRU) para hacer espacio a un nuevo bloque — recién ahí, si estaba dirty, se escribe de vuelta a la RAM antes de sobrescribirla.

Especificación de Datos

```
const TAMANO_RAM: usize = 256;

const BLOQUE_BYTES: usize = 4;

const CANTIDAD_CONJUNTOS: usize = 4;

#[derive(Debug, Clone, Default)]

pub struct LineaCache {
```

```

    pub tag: u8,
    pub valido: bool,
    pub dirty_bit: bool,
    pub datos: [u8; BLOQUE_BYTES],
    pub ultimo_acceso: u64,
}
pub struct ConjuntoCache {
    pub vias: [LineaCache; 2],
}
#[derive(Debug, Default)]
pub struct EstadisticasCache {
    pub hits: u64,
    pub misses: u64,
    pub desalojos_dirty: u64, // cuántas veces tuvo que escribir a RAM por desalojo
}
pub struct ControladorMemoria {
    pub ram: [u8; TAMANO_RAM],
    pub cache: [ConjuntoCache; CANTIDAD_CONJUNTOS],
    pub contador_ciclos: u64,
    pub estadisticas: EstadisticasCache,
}

```

Tareas a Implementar

Tarea 2.1 — Decodificación de direcciones

```

impl ControladorMemoria {
    /// Devuelve (tag, indice, offset) a partir de una dirección de 8 bits.
    pub fn decodificar_direccion(&self, direccion: u8) -> (u8, usize, usize) {
        todo!()
    }
}

```

```

    /// Operación inversa: reconstruye la dirección base del bloque
    /// (offset = 0) a partir de tag e índice. La vas a necesitar para
    /// saber a qué dirección de RAM corresponde una línea al desalojarla.
    pub fn reconstruir_direccion_base(&self, tag: u8, indice: usize) -> u8 {
        todo!()
    }
}

```

Tarea 2.2 — Búsqueda de vía (hit) y selección de víctima (LRU)

```

impl ControladorMemoria {
    /// Busca, dentro del conjunto dado, una vía válida cuyo tag coincida.
    /// Devuelve el índice de vía (0 o 1) si hay hit.
    fn buscar_via_hit(&self, indice_conjunto: usize, tag: u8) -> Option<usize> {
        todo!()
    }

    /// Elige qué vía desalojar dentro de un conjunto (para el caso de un miss).
    /// Regla de desempate obligatoria: si alguna vía tiene `valido == false`,
    /// esa se prefiere SIEMPRE antes de comparar `ultimo_acceso` (no tiene sentido
    /// desalojar una línea útil si hay una vía libre sin usar).
    /// Si ambas están ocupadas, se elige la de menor `ultimo_acceso` (la menos
    /// recientemente usada).
    fn elegir_via_victima(&self, indice_conjunto: usize) -> usize {
        todo!()
    }
}

```

Tarea 2.3 — Manejo de Miss (traer bloque + desalojo con write-back)

```
impl ControladorMemoria {  
    /// Maneja un cache miss: elige víctima, la desaloja escribiendo a RAM  
    /// si estaba dirty, trae el nuevo bloque desde RAM, y actualiza la línea.  
    /// Devuelve el índice de vía donde quedó cargado el bloque.  
    fn manejar_miss(&mut self, indice_conjunto: usize, tag: u8) -> usize {  
        todo!()  
    }  
}
```

Requisitos:

- Si la vía elegida tenía `valido == true` y `dirty_bit == true`: reconstruir su dirección base (con su tag viejo, Tarea 2.1) y escribir sus 4 bytes de vuelta a `self.ram` antes de sobrescribir. Incrementar `estadisticas.desalojos_dirty`.
- Traer los 4 bytes nuevos desde `self.ram` a partir de la dirección base del bloque solicitado.
- Dejar la línea con tag actualizado, `valido = true`, `dirty_bit = false`.

Tarea 2.4 — Lectura pública

```
impl ControladorMemoria {  
    pub fn leer_byte(&mut self, direccion: u8) -> u8 {  
        todo!()  
    }  
}
```

Requisitos: decodificar, buscar hit; si hay hit, actualizar `ultimo_acceso` con `contador_ciclos`, incrementar `estadisticas.hits`, devolver `datos[offset]`. Si hay miss, incrementar `estadisticas.misses`, llamar a `manejar_miss`, actualizar `ultimo_acceso`, devolver `datos[offset]`.

Tarea 2.5 — Escritura pública

```
impl ControladorMemoria {  
    pub fn escribir_byte(&mut self, direccion: u8, dato: u8) {  
        todo!()  
    }  
}
```

Requisitos: mismo flujo de hit/miss que la lectura, pero en vez de devolver el byte, **modificá** **datos[offset] = dato** en la línea correspondiente y marcá `dirty_bit = true`. Nunca escribas directamente en `self.ram` desde esta función — esa es exactamente la esencia de write-back.

Tarea 2.6 — Reporte de estadísticas

```
impl EstadisticasCache {  
    pub fn tasa_de_aciertos(&self) -> f64 {  
        todo!()  
    }  
}
```

Tests Obligatorios

1. **test_miss_luego_hit:** leé una dirección (miss esperado), volvé a leer la misma dirección (hit esperado). Verificá `estadisticas.misses == 1` y `estadisticas.hits == 1`.
2. **test_write_back_al_desalojar:** escribí un byte en una dirección (queda dirty en caché, sin tocar la RAM todavía — verificalo comparando contra el valor original en `self.ram`). Forzá un desalojo de esa línea (accediendo a 2 direcciones más con el mismo índice de conjunto para llenar las 2 vías y luego una tercera). Verificá que, tras el desalojo, `self.ram` en esa dirección **sí** tiene el valor escrito.
3. **test_lru_desaloja_la_correcta:** ocupá las 2 vías de un conjunto con 2 direcciones distintas (mismo índice, distinto tag). Accedé de nuevo a la primera (para "refrescar" su `ultimo_acceso`). Accedé a una tercera dirección con el mismo índice. Verificá que la vía desalojada fue la **segunda** (la que no se refrescó), no la primera.
4. **test_offset_correcto:** escribí 4 bytes en las 4 posiciones de un mismo bloque (mismo tag e índice, offsets 0 a 3) con valores distintos, y verificá que cada lectura posterior devuelve el valor correcto en su offset — que no se estén pisando entre sí.

5. **test_preferencia_via_libre_sobre_lru**: con una sola vía ocupada y la otra nunca usada (`valido == false`), forzará un miss en ese conjunto. Verificá que la línea ocupada **no** se desalojó (la vía libre fue la elegida), aunque su `ultimo_acceso` sea mayor a 0.
6. **test_tasa_de_aciertos**: con una secuencia conocida de hits/misses, verificá que `tasa_de_aciertos()` devuelve el porcentaje correcto.

Integración Final: Sistema Completo

Una vez que ambos proyectos tienen sus tests pasando por separado:

1. Completá la Tarea 1.5 (`ejecutar_mem`) para que use `ControladorMemoria::leer_byte/escribir_byte` de verdad.
2. Escribí un `main.rs` que:
 - Defina un programa de ejemplo con al menos un Load-Use Hazard y al menos un STORE.
 - Ejecute el pipeline ciclo a ciclo, imprimiendo el estado con tu `Display` (Tarea 1.8).
 - Al finalizar, imprima las estadísticas de caché (`hits`, `misses`, `tasa_de_aciertos`, `desalojos_dirty`).

Estructura de Entrega Sugerida

```
arquitectura-cpu-rust/  
├─ Cargo.toml           # workspace  
├─ README.md            # diagramas del pipeline, explicación de riesgos,  
  capturas de la demo  
├─ cpu-pipeline/  
  │ └─ src/lib.rs  
├─ cache-controller/  
  │ └─ src/lib.rs  
└─ sistema-integrado/  
  └─ src/main.rs
```

Criterios de Aceptación (Definition of Done)

- [] Los 5 tests del Proyecto 1 pasan.
- [] Los 6 tests del Proyecto 2 pasan.

- [] cargo clippy --workspace -- -D warnings sin errores.
- [] El main.rs integrado corre un programa con hazard + store/load real y muestra la traza completa.
- [] El README explica, con tus palabras, qué es un Load-Use Hazard y por qué el write-back mejora el rendimiento frente a write-through.

Extensiones Finales

Estas tareas cierran los huecos que quedan para considerar el TP terminado a nivel portfolio. Se dividen en **obligatorias** (cierran comportamiento que hoy falta) y **opcionales de alto impacto** (elevan el nivel de la demo/README, recomendadas si tenés tiempo).

Obligatorias

Tarea 3.1 — flush() en el controlador de caché

```
impl ControladorMemoria {
    /// Recorre todas las líneas de todos los conjuntos y, para cada una que
    /// esté válida Y dirty, escribe su contenido a RAM (usando
    /// reconstruir_direccion_base) y limpia el dirty_bit a false.
    /// A diferencia de un desalojo, la línea NO se invalida: sigue
    /// disponible en caché, solo queda sincronizada con la RAM.
    pub fn flush(&mut self) {
        todo!()
    }
}
```

Por qué importa: hasta ahora, la única forma de que un dato "sucio" llegue a la RAM es que lo desaloje otro acceso. flush() simula lo que hace un sistema operativo real antes de un cambio de contexto o un apagado ordenado — sincronizar todo a demanda, sin depender de que ocurran más accesos.

Test obligatorio — test_flush_sincroniza_sin_invalidar: escribí un byte (queda dirty, sin tocar RAM). Llamá a flush(). Verificá dos cosas: (1) la RAM ahora tiene el valor correcto, y (2) la línea sigue con valido == true y ahora dirty_bit == false (no se invalidó ni se perdió el contenido

de la caché).

Tarea 3.2 — Reporte de rendimiento combinado

```
/// Une las métricas de ambos proyectos en un solo reporte con sentido
/// de rendimiento real, no solo contadores sueltos.
pub fn reporte_rendimiento(
    cpu: &CpuSegmentada,
    mem: &ControladorMemoria,
    frecuencia_mhz: f64,
) -> String {
    todo!()
}
```

Requisitos del contenido del reporte:

- **Ciclos totales** (cpu.contador_ciclos).
- **CPI (Cycles Per Instruction)**: ciclos totales dividido instrucciones completadas (necesitás llevar un contador de instrucciones que efectivamente llegaron a WB — si no lo tenés, agregalo ahora a CpuSegmentada).
- **Tasa de aciertos de caché** (mem.estadisticas.tasa_de_aciertos()).
- **Tiempo estimado de ejecución en nanosegundos**: $(\text{ciclos_totales} / (\text{frecuencia_mhz} * 1_000_000.0)) * 1_000_000_000.0$.

Por qué importa: hoy tenés métricas de CPU y de caché en cajones separados. Esta función es la que un ingeniero real reporta al final de una simulación — "este programa tardó X ns a Y MHz con Z% de aciertos de caché" es una conclusión con sentido de negocio, no solo un contador aislado.

Test obligatorio — test_reporte_incluye_metricas_esperadas: corré un programa breve y conocido de antemano (con stalls e instrucciones fijas), y verificá que el string devuelto contiene los valores numéricos correctos para ciclos, CPI y tasa de aciertos (podés usar `assert!(reporte.contains(...))` o parsear el número si preferís ser más estricto).

Tarea 3.3 — Programa de prueba: forwarding en cadena sin stalls

Agregá el siguiente programa como caso de test (no requiere código nuevo en la implementación, solo un test que lo ejercite):

```
ADD R1, R2, R3      ; R1 = R2 + R3
ADD R2, R1, R0      ; depende de R1 (forwarding EX/MEM)
SUB R3, R2, R1       ; depende de R2 (forwarding EX/MEM, un ciclo después)
```

Test obligatorio — test_forwarding_en_cadena_sin_stalls: ejecutá este programa y verificá que **ningún ciclo** requiere stall (detectar_load_use_hazard nunca devuelve true durante toda la ejecución), a pesar de que cada instrucción depende de la anterior. Contrastalo mentalmente (y dejalo en un comentario) con test_stall_no_pierde_la_instruccion_load: misma "dependencia de datos" entre instrucciones consecutivas, pero acá el forwarding la resuelve sin frenar el pipeline porque el dato productor **no** viene de un LOAD. Este es el concepto que distingue por qué el Load-Use Hazard es el único caso que obliga a detener la CPU.

Opcionales de Alto Impacto

Tarea 3.4 — Simulación dirigida por traza (trace-driven)

```
pub enum OperacionMemoria {
    Leer(u8),
    Escribir(u8, u8),
}

/// Ejecuta una secuencia arbitraria de operaciones de memoria contra el
/// controlador, sin pasar por la CPU. Útil para comparar patrones de acceso.
pub fn ejecutar_traza(mem: &mut ControladorMemoria, traza: &[OperacionMemoria]) {
    todo!()
}
```

Sugerencia de demo para tu main.rs: generá dos trazas de, por ejemplo, 50 operaciones cada una:

- Una con **acceso secuencial** (direcciones 0, 1, 2, 3, ... en orden).
- Una con **acceso disperso/aleatorio** (direcciones saltando sin patrón, por ejemplo alternando entre bloques muy separados).

Corré ejecutar_traza con cada una sobre un ControladorMemoria nuevo, e imprimí tasa_de_aciertos() de ambas una al lado de la otra. Deberías ver una diferencia notable — el acceso secuencial se beneficia de la localidad espacial (varios bytes del mismo bloque de 4 se piden seguidos), mientras que el disperso casi no reutiliza bloques cargados. Esta comparación con números reales es mucho más convincente en un README que solo tests unitarios pasando.

Tarea 3.5 — Predicción estática de saltos con penalidad (si ya hiciste la Tarea 1.9 JUMP)

Si implementaste el riesgo de control del stretch goal original, extendelo:

```
pub struct EstadisticasCpu {  
    pub saltos_totales: u64,  
    pub saltos_mal_predichos: u64,  
    pub ciclos_perdidos_por_flush: u64,  
}
```

Requisito: asumí una predicción estática de "**salto no tomado**" (la CPU sigue buscando instrucciones secuencialmente, como si el JUMP no fuera a ejecutarse). Cuando el JUMP se resuelve en EX y **sí** correspondía saltar, contabilizá la misprediction, hacé el flush de if_id/id_ex como ya tenías, y sumá los ciclos perdidos (2, por las 2 instrucciones descartadas) a ciclos_perdidos_por_flush.

Test sugerido: un programa con un JUMP que efectivamente salta, verificando que saltos_mal_predichos == 1 y que las instrucciones que estaban mal encaminadas nunca llegan a escribir en el banco de registros.

Tarea 3.6 — Reflexión técnica en el README

No es código, pero es lo primero que lee cualquier persona técnica que revise tu repositorio. Agregá una sección "Decisiones de Diseño" respondiendo, con tus propias palabras y basándote en lo que implementaste (no copies definiciones genéricas de internet):

1. ¿Por qué **write-back** es más eficiente que **write-through** en términos de tráfico hacia la

RAM? ¿Qué riesgo se acepta a cambio (pensá en qué pasaría si el sistema pierde energía con líneas dirty sin sincronizar)?

2. Tu caché es **write-allocate** (un miss de escritura trae el bloque completo a caché antes de escribir). ¿En qué escenario de acceso a memoria convendría lo contrario (no-write-allocate, escribir directo a RAM sin cachear)?
3. ¿Por qué el Load-Use Hazard es el único caso, de todos los riesgos de datos, que el forwarding no puede resolver por sí solo?

Criterios de Aceptación Actualizados

- [] Los 5 tests originales del Proyecto 1 pasan.
- [] Los 6 tests originales del Proyecto 2 pasan.
- [] test_flush_sincroniza_sin_invalidar pasa (Tarea 3.1).
- [] test_reporte_incluye_metricas_esperadas pasa (Tarea 3.2).
- [] test_forwarding_en_cadena_sin_stalls pasa (Tarea 3.3).
- [] cargo clippy --workspace -- -D warnings sin errores.
- [] El main.rs integrado corre un programa con hazard + store/load real, muestra la traza completa, y termina imprimiendo el reporte de rendimiento combinado.
- [] El README incluye la sección "Decisiones de Diseño" con las 3 reflexiones respondidas.
- [] (*Opcional*) la demo de traza secuencial vs. dispersa está incluida y sus resultados comentados en el README.